INFORME TÉCNICO



Trabajo Práctico Grupal: Software embarcado de una nave interestelar tripulada

Asignatura: Programación C

Grupo: 16

Lenguaje: Java

1. INTRODUCCIÓN Y ALCANCE

En este informe detallamos el diseño, la arquitectura y la implementación de nuestro sistema de simulación espacial. La idea principal del trabajo fue modelar un entorno donde convivan de forma robusta naves espaciales, asistentes de a bordo automatizados, tripulaciones con roles específicos y distintas misiones operativas.

Para lograrlo, nos basamos en una serie de reglas y restricciones de diseño bien marcadas:

- Separar por completo el control del universo o entorno general de la lógica interna y el funcionamiento de las naves.

- Asegurar que las naves no tomen decisiones por su cuenta. Todas las acciones y operaciones pasan por la supervisión de asistentes de comando independientes.
- Modelar el motor Warp y sus transiciones de estado de forma estricta, evitando caer en la clásica cadena de condicionales anidados (if/else).
- Armar un sistema de misiones estandarizado de 4 etapas que primero verifique los recursos y las condiciones de vuelo antes de alterar el estado de la nave.
- Mantener un registro histórico inmutable de los sucesos a través de una bitácora de a bordo.
- Diseñar una tripulación flexible, permitiendo agregar características a las especies o roles de forma dinámica.

2. MODELADO DE COMPONENTES Y PATRONES DE DISEÑO

2.1. Máquina de Estados del Motor Warp (State Pattern) Como teníamos prohibido usar condicionales anidados para cambiar las fases del motor, la mejor alternativa fue aplicar el patrón State.

- **Contexto:** `MotorWarp` contiene una referencia a `EstadoWarp` y le delega las operaciones principales como `prepararSalto()`, `saltar()` y `enfriar()`.
- **Estados Concretos:**
    1. **Disponible:** Permite llamar a `prepararSalto()`, haciendo que el motor pase al estado PreparandoSalto. Si se intenta otra cosa (como saltar de la nada), la acción se ignora, protegiendo la integridad del sistema.
    2. **PreparandoSalto:** Habilita el método `saltar()`, pasando a EnWarp.
    3. **EnWarp:** Representa el viaje a hipervelocidad. Como temporalmente no computamos el paso continuo del tiempo, al terminar el salto pasa directamente a Disponible, reconociendo la fase de Enfriamiento de manera conceptual.
    4. **Enfriamiento:** Diseñado para la disipación térmica previa a volver a estar Disponible.

Ventaja: Si el día de mañana queremos agregar un estado (por ejemplo, `EnReparacion`), solo sumamos una clase nueva sin tener que tocar el código original de `MotorWarp`.

2.2. Ciclo de Ejecución de Misiones (Template Method / Command) El sistema exigía que toda misión cumpliera sí o sí con cuatro etapas en un orden estricto. Para no repetir código, armamos una clase base abstracta `Mision`:

1. **Esqueleto (Algoritmo base):** Un método principal define el flujo inalterable:

    - *Paso 1 (Validación):* Le consulta al asistente si la nave está Disponible y si le alcanzan los recursos. Si algo falla, se aborta y va a la bitácora.
    - *Paso 2 (Consumo):* Descuenta la energía y el combustible necesarios y manda a preparar el salto.
    - *Paso 3 (Salto Warp):* Ejecuta el salto espacial.
    - *Paso 4 (Finalización):* Cierra la operación e imprime los carteles correspondientes.
2. **Especializaciones:** Las misiones concretas (`Mision01`, `Mision02`, `Mision03`) solo redefinen cuánto consumen y el cartel que muestran al final en el paso 4. Toda la secuencia de vuelo queda intacta.

2.3. Creación de Naves (Factory Method / Simple Factory) Para no acoplar el entorno externo con la creación específica de las naves, usamos `Fabricacion_Nave`:

- Centraliza la creación de `Nave_Carguero`, `Nave_Combate` y `Nave_Exploradora`.
- Oculta las configuraciones iniciales (cuánto combustible o energía traen de base).
- Lanza una excepción (`TipoNaveInvalidoException`) si alguien pide un tipo de nave que no existe, protegiéndonos de estados inconsistentes.

2.4. Registro de Eventos e Inmutabilidad (Bitácora) La clase `Evento` encapsula la hora exacta (timestamp) y el detalle de lo que pasó.

- **Inmutabilidad:** Los atributos son privados y finales, no hay "setters". Lo que se registra, queda fijo para siempre.
- La bitácora vive dentro del Asistente como una lista. Cada vez que se usa el motor, se gastan recursos o se hace una misión, se genera un evento nuevo para la auditoría de a bordo.

2.5. Modelado Dinámico de la Tripulación (Decorator Pattern) Para manejar a los tripulantes queríamos sumarles habilidades según su raza o planeta de origen sin terminar armando un árbol gigante de subclases combinadas (ej. `CapitanMarciano`, `TenienteVulcano`). Fuimos por el patrón Decorator:

- **Componente Base:** `Tripulante` define el contrato del personal.
- **Componentes Concretos:** Son los rangos funcionales (Capitán, Teniente, Alferez, Consejero).
- **Decoradores:** `DecoratorTerricola`, `DecoratorMarciano` y `DecoratorVulcano` "envuelven" al tripulante en tiempo de ejecución para agregarle sus características gentilicias sin modificar su clase original.

3. ANÁLISIS DE PRINCIPIOS SOLID

- **Responsabilidad Única (SRP):** Separamos muy bien las tareas. `MotorWarp` solo se enfoca en la propulsión, `Recursos` lleva los balances numéricos, `Evento` es un registro de historia simple, y `Asistente` es el único gestor intermedio.
- **Abierto/Cerrado(OCP):** Podemos inventar una `Mision04` o sumar un nuevo estado de motor creando clases nuevas, sin tocar una sola línea del código base.
- **Sustitución de Liskov (LSP):** Cualquier misión concreta se puede usar sin problemas donde se espera una `Mision` genérica. Lo mismo pasa con las implementaciones de `EstadoWarp`.
- **Segregación de Interfaz (ISP):** Mantuvimos los contratos chicos. No forzamos a ninguna clase a implementar métodos que no tienen sentido para su funcionamiento.
- **Inversión de Dependencias (DIP):** Todo el sistema se apoya en abstracciones (`Nave`, `Asistente`, `EstadoWarp`), haciendo que el código no dependa de implementaciones puntuales de bajo nivel.

4. CONCLUSIONES

El desarrollo de este TP nos permitió bajar a tierra los requerimientos de la consigna logrando una arquitectura muy limpia y fácil de mantener:

- El patrón **State** resolvió excelente la complejidad del motor warp, sacando del medio los condicionales.
- El **Template Method** nos aseguró que nadie se pueda saltear accidentalmente los pasos de validación de una misión.
- El uso del **Asistente** como puente único fue clave para encapsular el acceso a los recursos de las naves.
- El patrón **Decorator** le dio a la tripulación una flexibilidad enorme para el futuro.

En conclusión, la clara separación de responsabilidades y la aplicación de los principios SOLID garantizan que, el día de mañana, si queremos escalar la simulación o ampliar lo que hace la `ClasePrueba`, no vamos a tener que romper ni modificar el código central del dominio espacial.